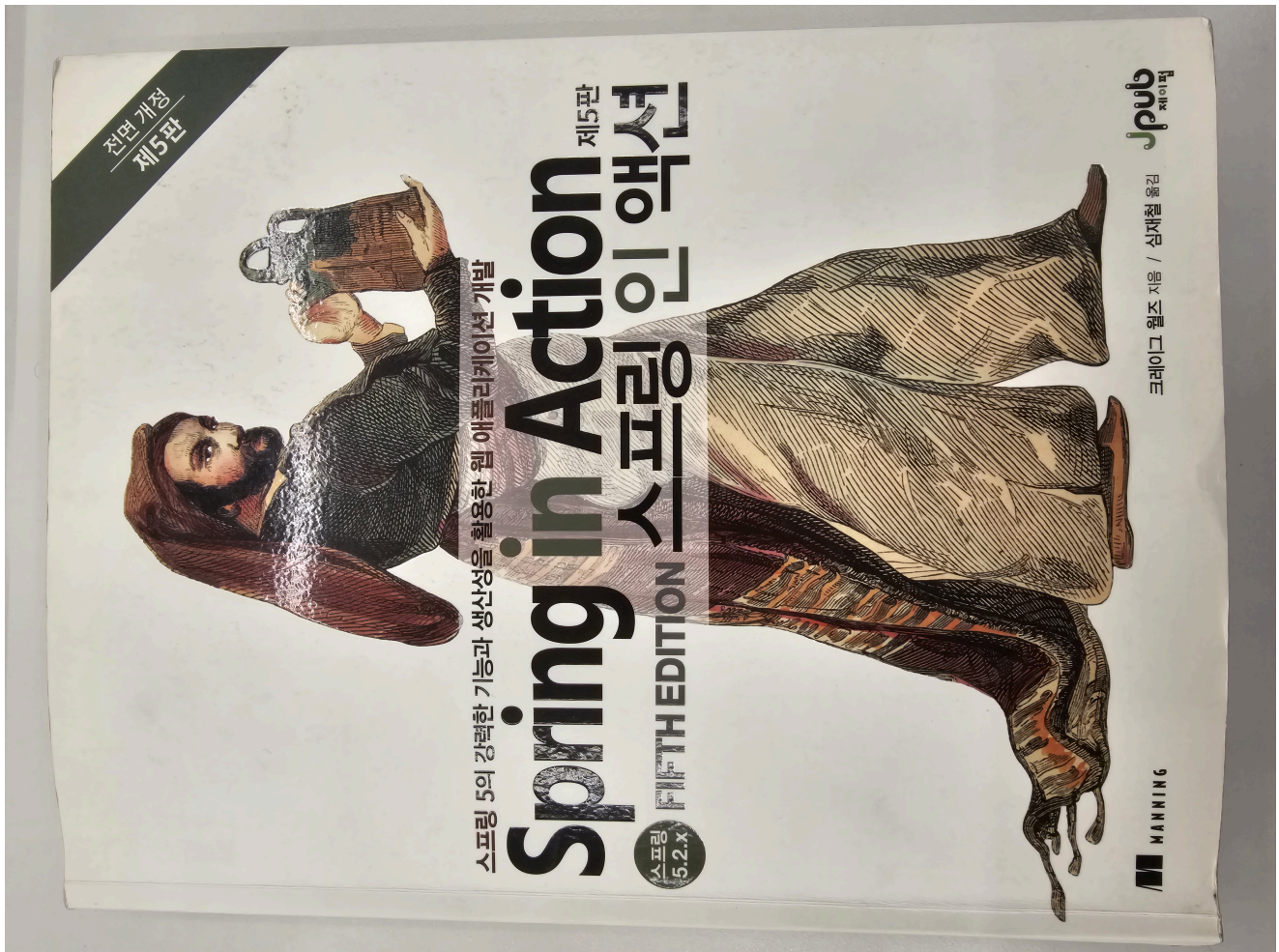


Spring boot - 5

■ Spring in Action 5판 기반 스프링 부트 공부 필기

참고

- 책: *Spring in Action* 제5판 / 크레이그 윌즈 지음, 심재철 옮김
- 코드 및 예시는 책 내용을 기반으로 함
- 나의 학습 환경: Java 17, IntelliJ
- PDF 다운로드: [<https://plusb3b.kr/pdf-download-page/index.html>]
- 책의 공식 GitHub: <https://github.com/Jpub/SpringInAction5>



🌱 Spring Boot 구성 속성 가이드

Spring Security는 중요한 주제이지만 학습량이 방대하므로, 먼저 Spring Boot의 구성 속성 사용법부터 차근차근 알아보겠습니다.

📌 자동 구성이란?

Spring Boot의 **자동 구성(Auto-Configuration)**은 Spring Application 개발을 단순화합니다.

구성 속성은 JVM 시스템 속성, 명령행 인자, 환경 변수 등 다양한 소스에서 설정 가능하며, 이는 **Bean의 속성**으로 주입되어 활용됩니다.

🌸 5.1 자동 구성 세부 조정

Spring에서는 서로 다르지만 관련 있는 두 가지 구성 형태를 제공합니다:

🌸 1. Bean Wiring (빈 연결)

정의:

Spring Application Context에서 Bean으로 생성된 애플리케이션 컴포넌트들과 이들 간의 의존성 주입 방식을 설정하는 구성

기능:

- `@Component`, `@Service`, `@Repository`, `@Controller` → 자동 빈 등록
- `@Autowired`, `@Inject`, `@RequiredArgsConstructor` → 자동 주입

핵심 개념:

- **Bean**: 연결되는 대상 객체
- **Spring Container**: Bean의 생성, 주입, 생명주기 담당

🔗 2. Property Injection (속성 주입)

정의:

설정 파일에 정의된 값을 객체의 필드나 생성자, setter에 자동 주입하는 방식

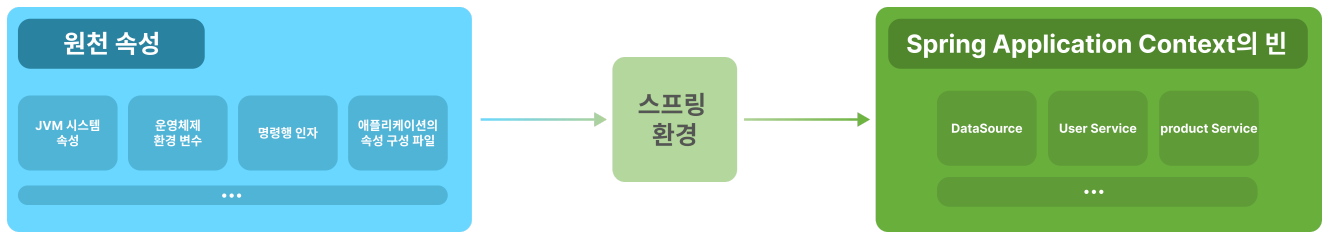
이 구성은 XML, Java 기반 구성 방식 모두에서 사용 가능합니다.

🌐 5.1.1 스프링 환경 추상화 이해하기

****환경 추상화(Environment Abstraction)****는 JVM, 명령행 인자, OS 환경변수, 설정 파일 등의 다양한 원천 속성을 하나로 통합해 관리합니다.

🔍 용어 정리

용어	설명
Spring Container	BeanFactory, ApplicationContext 등 Bean의 생성과 관리 담당
ApplicationContext	Spring Container의 확장. 이벤트 처리, 메시지 처리 등 추가 기능 제공



💬 비유로 이해하기

"속성의 근원으로부터 원천 속성을 가져온다"

- 속성의 근원: 병사의 출신지
- 환경: 훈련소/행정실
- 속성: 병사 정보 (이름, 나이 등)

🔧 server.port 설정하기

서버 포트 변경 방법:

- application.properties

```
server.port=9090
```

- application.yml

```
server:  
  port: 9090
```

- 명령행:

```
java -jar taco.jar --server.port=9090
```

- 환경변수:

```
export SERVER_PORT=9090
```

📁 5.1.2 데이터 소스 구성

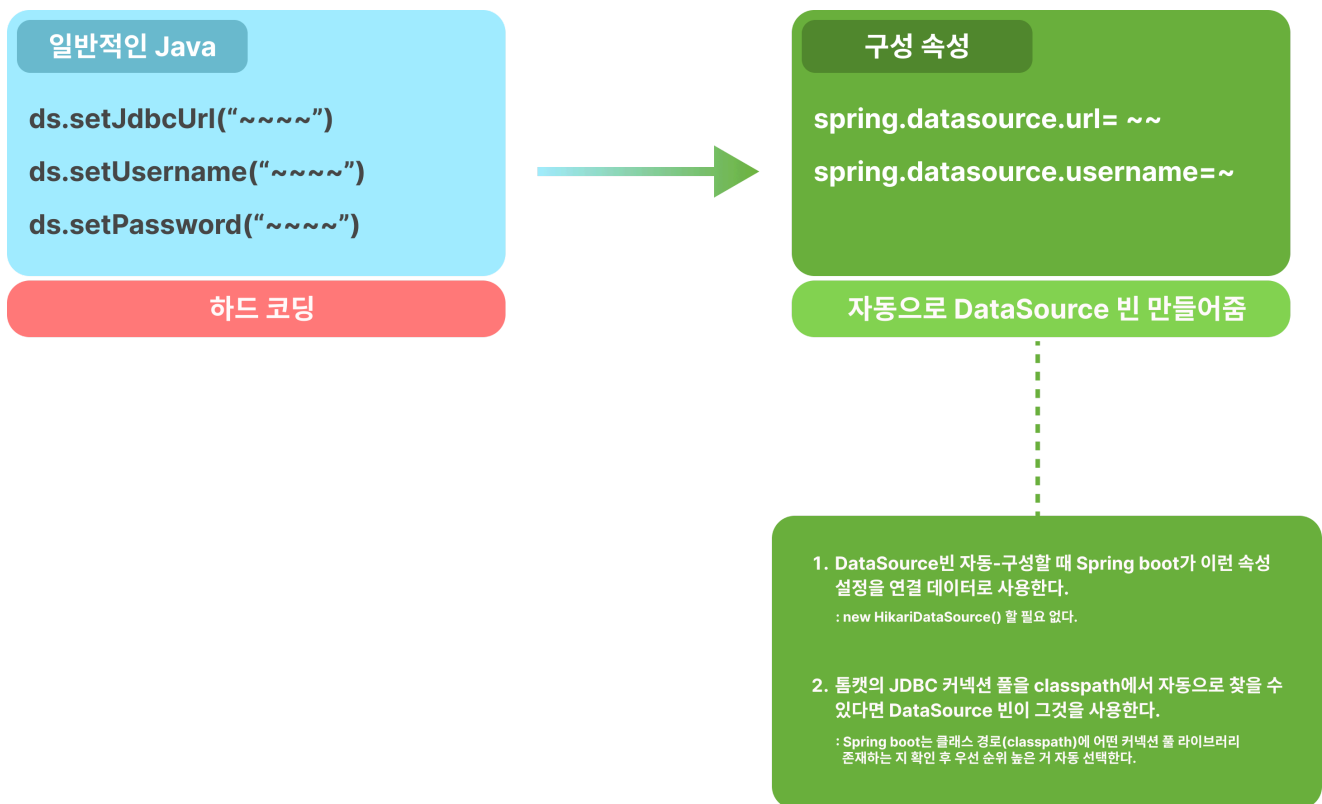
Spring Boot는 설정만으로 DB 연결 가능:

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/tacocloud
    username: your_username
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver
```

✓ 커넥션 풀 자동 선택 우선순위

1. HikariCP (기본)
2. Tomcat JDBC Pool
3. Commons DBCP2

📌 커넥션 풀 이미지



📄 SQL 스크립트 자동 실행

```
spring:
  datasource:
    schema:
      - taco-schema.sql
    data:
      - ingredients.sql
```

📡 JNDI 방식

```
spring:
  datasource:
```

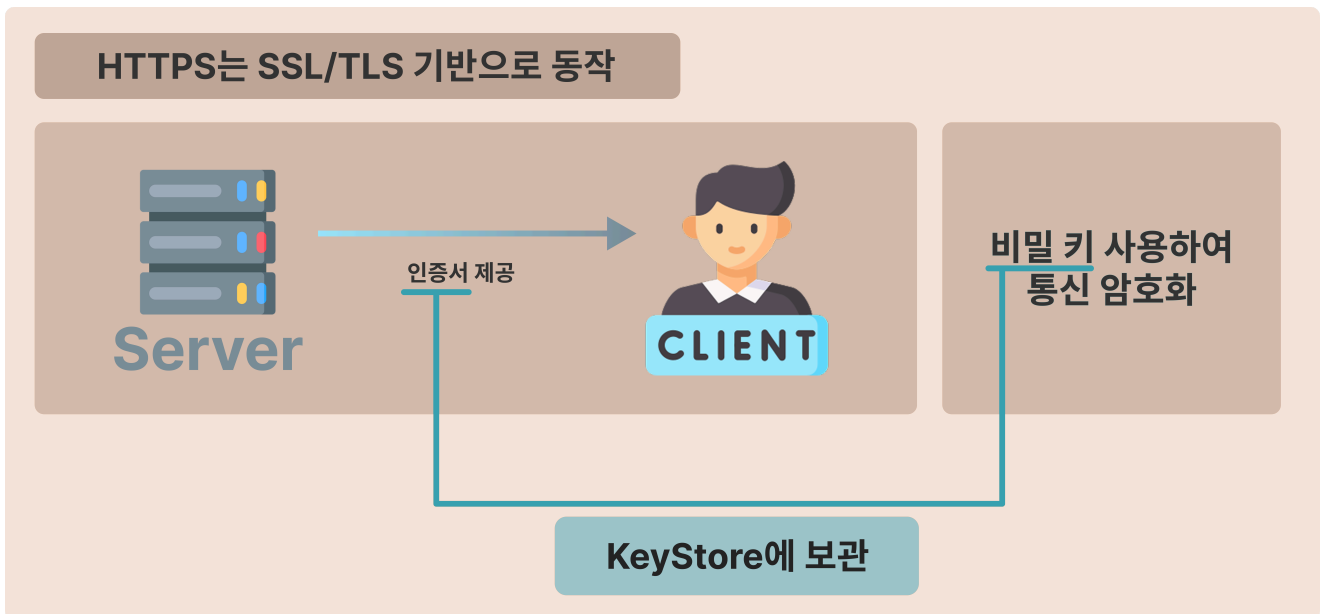
jndi-name: java:/comp/env/jdbc/tacoCloudDS

🔒 5.1.3 HTTPS 설정하기

🔑 keytool을 이용한 keystore 생성

```
keytool -keystore mykeys.jks -genkey -alias tomcat -keyalg RSA
```

🖼️ 관련 이미지:



📄 설정 예시:

설정 예시

server:

port: 8443

ssl:

- 1 **key-store : file//path/to/mykeys.jks**
- 2 **key-store-password: letmein**
- 3 **key-password: letmein**

1

윈도우일 경우 “file:///C:/mykeys.jks” 로 써야한다

로컬 파일

Java KeyStore 파일

2

키스토어 파일 전체를 열기 위한 파일

keytool로 키스토어 만들 때 입력했던 비밀번호 설정

3

키스토어 내부의 개별키에 접근하기 위한 비밀번호

server.port 속성은 8443으로 설정되었다. 개발용 HTTPS 서버에 많이 사용된다.

server.ssl.key-store 속성은 키스토어 파일이 생성된 경로로 설정되어야한다.

운영체제의 파일 시스템에서 키스토어 파일을 로드하기 위해 file://를 URL로 지정

server.ssl.key-store-password와 server.ssl.key-password 속성에는 키스토어를 생성할 때 지정했던 비밀번호를 설정한다

애플리케이션 JAR 파일에 키스토어 파일 넣는 경우는 classpath: 를 URL 지정

5.1.4 로깅 구성

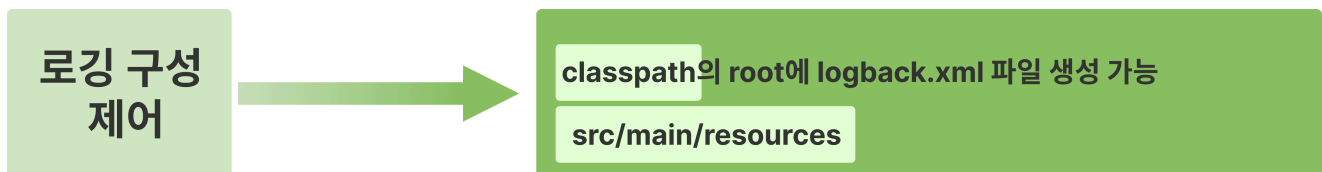
Spring Boot 기본 로깅: Logback + INFO 수준

```
logging:
  file:
    path: /var/logs/
    name: TacoCloud.log
  level:
    root: WARN
    org.springframework.security: DEBUG
```

로깅 레벨 종류

- TRACE > DEBUG > INFO > WARN > ERROR > OFF

 구조 이미지



5.1.5 속성 값 참조하기

```
greeting:
  welcome: You are using ${spring.application.name}
```

5.2 사용자 정의 구성 속성

구성 속성 정의 예시

```
taco:
  orders:
    pageSize: 20
```

Java 클래스:

```
@Component
@ConfigurationProperties(prefix = "taco.orders")
```

```
@Validated
@Data
public class OrderProps {
    @Min(5)
    @Max(25)
    private int pageSize = 20;
}
```

메타데이터 설정 (IDE 자동완성용)

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

resources/META-INF/additional-spring-configuration-metadata.json :

```
{
  "properties": [
    {
      "name": "taco.orders.page-size",
      "type": "java.lang.Integer",
      "description": "한 페이지에 표시할 주문 수를 설정한다."
    }
  ]
}
```

5.3 프로파일로 구성 분리하기

환경별 설정 분리를 위해 **application-prod.yml** 등으로 설정 가능

```
logging:
  level:
    tacos: DEBUG

---

spring:
  profiles: prod
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
logging:
  level:
    tacos: WARN
```

프로파일 활성화 방법

- 명령행:

```
java -jar app.jar --spring.profiles.active=prod
```

- 환경변수:

```
export SPRING_PROFILES_ACTIVE=prod,qa
```

5.3.3 조건별 Bean 등록

```
@Bean
@Profile("prod")
public MyService myService() {
    return new MyService();
}

@Bean
@Profile({"dev", "qa"})
public CommandLineRunner dataLoader(...) {
    // 개발용 로딩
}

@Bean
@Profile("!prod")
public CommandLineRunner localLoader(...) {
    // 운영 제외용
}
```

마무리

- Spring Boot 구성 속성은 다양한 환경에 유연하게 대응 가능
- @ConfigurationProperties 와 프로파일을 적절히 사용하면 설정 관리가 쉬워짐
- 자동 구성 + 사용자 정의 속성 + 로깅 + DB 설정까지 한 눈에 이해!

다음에는 Spring Security 주제로 이어서 정리해보겠습니다 🙌

[springboot-5.pdf](#)

[Spring boot - 3](#)